

Auditable Data-Centric Unlearning in Retrieval-Augmented and Fine-Tuned Foundation Models

Author Names Omitted for Review

Abstract—Deletion requests in foundation-model applications are difficult to audit because user data can influence behavior through multiple data-bearing artifacts: raw documents, chunks, embeddings, vector indexes, synthetic summaries, fine-tuning records, adapters, caches, and generated traces. Existing machine-unlearning evaluations often focus on model weights or exact memorization probes, leaving retrieval-time dependence, paraphrased leakage, and derivative artifacts under-tested. This paper proposes ADCU, an auditable data-centric unlearning framework for retrieval-augmented and fine-tuned foundation-model systems. ADCU models deletion as a pipeline-level data operation over a provenance graph, defines a deletion-risk metric that combines direct leakage, paraphrased leakage, retrieval dependence, counterfactual answer dependence, extraction risk, and watermark hits, and implements a valuation-guided black-box audit algorithm with confidence-bounded pass/fail/escalate decisions. We introduce ADCU-Bench, a benchmark protocol covering RAG deletion, fine-tuning record unlearning, and hybrid RAG-to-fine-tuning derivative retention. A synthetic pilot and attack suite show that exact-match, canary-only, and retriever-only audits miss important residual-dependence channels, while ADCU detects all configured deletion failures with zero false alarms in the pilot setting. The results position deletion verification as a data engineering problem for foundation-model systems, complementing model-centric unlearning algorithms with measurable audit evidence.

Index Terms—Machine unlearning, retrieval-augmented generation, foundation models, data provenance, data valuation, deletion auditing.

1 INTRODUCTION

Foundation-model applications are no longer single models queried in isolation. A deployed assistant, analyst tool, or enterprise chatbot may combine retrieval-augmented generation (RAG), dense and lexical indexes, rerankers, prompt templates, supervised fine-tuning, LoRA adapters, synthetic-data generation, feedback traces, caches, monitoring logs, and human-facing citations. RAG has become a standard way to connect language models to external knowledge sources [1], [2], [3], [4], while instruction tuning and adapter-based specialization make private or domain-specific records part of model behavior. This architecture improves utility, but it also makes deletion requests technically ambiguous: deleting a raw document, account record, or support transcript does not necessarily remove the chunks, embeddings, summaries, synthetic question-answer pairs, cache entries, adapters, or evaluation traces that were derived from it.

This ambiguity is becoming a central data-management problem for foundation-model systems. Privacy and governance regimes motivate data erasure and deletion accountability [5], [6], and recent foundation-model challenge work identifies private data use, continual learning, unlearning, watermarking, and efficiency as important open problems [7]. Machine unlearning provides the algorithmic foundation for making systems forget selected data [8], [9], [10], [11]. However, in modern LLM and RAG deployments, the deletion problem is not confined to the parameter vector. A post-deletion system can pass a model-centric unlearning test while still retrieving a stale chunk, citing a shadow copy, answering from a synthetic derivative, or paraphrasing a fact learned through an adapter.

The difficulty is that deletion is a behavioral property of a pipeline, not merely a storage operation or a weight update. Existing LLM-unlearning benchmarks and methods increasingly study forgetting and retained utility for adapted language models [12], [13], [14], [15], [16]. Separately, extraction and membership-inference work shows that language models can reveal training data or expose membership signals through black-box interaction [17], [18], [19], [20], [21], [22]. RAG evaluation studies retrieval quality, attribution, grounding, and context usage [23], [24], [25], [26]. These lines of work are all necessary, but none directly answers the operator’s audit question: after a deletion request, what measurable evidence shows that the deleted data and its downstream derivatives no longer influence retrieval, generation, or fine-tuned behavior?

We argue that deletion verification should be treated as an auditable data-centric systems problem. The relevant object is not only a model checkpoint; it is the collection of data-bearing artifacts that may preserve a deleted record’s influence. These artifacts include retriever documents, chunk boundaries, embedding rows, reranker candidates, prompt caches, summaries, synthetic examples, adapter updates, canaries, provenance tokens, and generated traces. A practical audit must therefore combine three ingredients. First, it needs provenance: which artifacts are downstream of the deletion target? Second, it needs prioritization: which deleted records had high influence or many derivatives and therefore deserve more audit budget? Third, it needs behavioral tests: does the post-deletion system still reproduce, retrieve, paraphrase, infer, or cite the deleted

information?

This paper proposes ADCU (*Auditable Data-Centric Unlearning*), a framework for deletion-risk auditing in retrieval-augmented and fine-tuned foundation-model systems. ADCU represents deletion targets and their downstream artifacts in a provenance graph; assigns audit priority using pre-deletion influence and provenance degree; generates direct, paraphrase, retrieval, counterfactual, extraction, trigger, and graph-pivot probes; and scores observed behavior across multiple residual-dependence channels. Instead of issuing a binary claim of perfect forgetting, ADCU returns a confidence-bounded pass, fail, or escalate decision under an operator-chosen tolerance. The goal is not to replace unlearning algorithms, certified removal mechanisms, differential privacy, or legal review [27], [11]. The goal is to provide measurable technical evidence about residual behavioral dependence in the data pipeline surrounding those mechanisms.

The core deletion-risk metric aggregates six evidence channels: direct leakage, paraphrased leakage, retrieval dependence, counterfactual answer dependence, extraction risk, and watermark or provenance-token hits. This design is motivated by known failure modes. Exact-match audits can miss paraphrased leakage; canary-only audits can fail when obvious markers are suppressed; membership tests can miss retrieval-only residues; and retriever-only audits can miss model-side memory or answer-side paraphrases. Recent work on RAG security and extraction further shows that leakage can arise through benign-looking queries, triggered behavior, and graph or retrieval pivots rather than only through direct requests for secrets [28], [29], [30], [31]. ADCU therefore treats deletion verification as a multi-channel audit rather than a single memorization test.

We instantiate the framework in ADCU-Bench, a reusable benchmark and artifact for deletion auditing. The benchmark covers four complementary settings. *RealRAG* uses public FEVER evidence as a retain corpus while injecting synthetic private deletion targets and downstream derivatives. *NaturalFEVER* treats public FEVER evidence passages themselves as deletion units, producing a natural-data deletion case with redacted outputs. *FineTuneSim* provides a controlled adapter-memory simulator for repeated unlearning stress tests. *HybridReal* combines RAG records and derivative artifacts that survive through summaries, caches, synthetic examples, or model-side behavior. We further add LoRA-SFT, cached PEFT/LoRA validations, pretrained LoRA margin analysis, TOFU-style public forget/retain evaluation, neural dense retrieval with MiniLM/E5/BGE-style retrievers, and a local cross-encoder-style reranking study.

Across the repeated submission-scale harness, ADCU detects configured deletion failures in RealRAG, NaturalFEVER, and HybridReal while preserving zero false alarms in the configured clean cases; it detects 97.2% of FineTuneSim failures where narrow baselines miss important channels. The advanced experiments show the same pattern in model-side and retrieval-side settings: retriever-only audits do not detect LoRA memory, exact matching is brittle under paraphrase, and canary-only tests fail when semantic facts remain without the marker. The point of these experiments is not that ADCU proves legal compliance

or absolute absence of influence. Rather, the results show that deletion audits become substantially more informative when they are provenance-aware, valuation-guided, and multi-channel.

This paper makes the following contributions:

- We formulate deletion verification for RAG and fine-tuned foundation-model systems as a pipeline-level data audit over provenance-linked artifacts, rather than as only model-weight unlearning.
- We define a deletion-risk metric that combines direct leakage, paraphrased leakage, retrieval dependence, counterfactual answer dependence, extraction risk, and watermark or provenance-token evidence.
- We present a valuation-guided black-box audit algorithm that allocates limited probe budgets toward high-risk deletion targets and returns confidence-bounded pass, fail, or escalate decisions.
- We introduce ADCU-Bench, a reusable benchmark spanning synthetic private deletion, natural public-evidence deletion, RAG retention, fine-tuning residues, hybrid derivative retention, dense retrieval, pretrained LoRA, and TOFU-style public forget/retain evaluation.
- We provide an attack and baseline suite showing where exact-match, canary-only, retriever-only, membership-inference, extraction-only, and influence-proxy audits fail, together with redacted case studies and artifact code for reproduction.

The remainder of the paper is organized as follows. Section 2 discusses related work in machine unlearning, RAG, data valuation, extraction, watermarking, and deletion governance. Section 3 defines the audit problem and threat model. Sections 4 and 5 present the deletion-risk metric and audit algorithm. Section 6 describes ADCU-Bench. Section 7 reports the experiments, baselines, ablations, and case studies. Sections 8 and 9 discuss limitations, artifact release, and reproducibility, and Section 10 concludes.

2 RELATED WORK

Machine unlearning. Machine unlearning studies how systems can remove the effect of selected training data after model deployment [8], [10], [9]. Foundation-model unlearning extends this challenge to large pretrained and adapted models, where exact retraining is often infeasible [13], [12], [14], [15]. Recent surveys emphasize that LLM unlearning must evaluate both forgetting and retained utility [16]. Our work differs by focusing on auditability across the data pipeline rather than proposing a new weight-update rule.

Retrieval-augmented generation. RAG systems condition generation on retrieved external documents [1], [2], [3], [4]. This improves factual grounding but introduces deletion risk: removing a source document from one index does not necessarily remove derived chunks, summaries, cached contexts, or fine-tuning examples generated from that source. Recent work studies attribution, context usage, source tracing, RAG evaluation fragmentation, and enterprise control points [23], [24], [25], [26]. ADCU treats these artifacts as first-class audit objects and adds deletion risk as an orthogonal audit axis.

Data valuation and influence. Data valuation methods estimate how much individual records contribute to model utility [32], [33], [34], [35]. Influence functions and related attribution methods estimate the effect of training examples on predictions [36], [37], [38], [39]. ADCU uses this line of work differently: valuation is used to prioritize audit effort after deletion requests.

Extraction, membership, canaries, and watermarking. Prior work has shown that language models can expose training data through extraction attacks [18], [17], [19]. Membership inference also provides evidence of training-data dependence [20], [21], [22]. Canaries and watermarks provide useful signals [40], [41], but canary-only audits can be defeated when the protected semantic fact remains after the obvious marker is suppressed. ADCU therefore combines watermark hits with semantic and retrieval-dependence evidence.

Privacy and deletion compliance. Deletion auditing is motivated by privacy rights and governance obligations such as data erasure, but technical audits should not overclaim legal compliance. Prior privacy work on differential privacy, memorization, certified removal, and deletion-as-control provides useful context [27], [5], [11], [6]. ADCU contributes measurable technical evidence that can coexist with formal mechanisms: when certified removal or DP continual release is available, ADCU can monitor pipeline drift and derivative artifacts outside the formal core.

Foundation-model data governance. Recent challenge work on federated foundation models identifies private data utilization, continual learning, unlearning, model watermarking, and efficiency as major open problems [7]. ADCU operationalizes these concerns for RAG and fine-tuned systems by defining measurable deletion-risk audits over provenance-aware data pipelines.

Membership and extraction baselines. Membership-style audits ask whether the system behaves differently on records that were present in training or adaptation data. Extraction-style audits ask whether prompting can recover sensitive spans. Both are useful but incomplete: membership tests may miss retrieval-only leakage, and extraction prompts may miss ordinary paraphrased answer dependence. Our benchmark therefore includes membership-inference, extraction, retriever-only, and influence-proxy baselines beside full ADCU.

RAG provenance and reranking. RAG pipelines often contain multiple ranking stages. A first-stage vector index can be clean while a reranker, cache, query expansion layer, or synthetic derivative reintroduces deleted facts. This motivates our DenseRAG extension, which evaluates lexical retrieval, dense SVD retrieval, and a local cross-encoder-style reranker under the same deletion audit interface.

RAG extraction, triggered leakage, and graph pivots. Recent RAG security work shows that leakage can occur through benign-looking extraction queries, runtime canary suppression, and graph-expansion pivots rather than only through obvious jailbreak prompts [28], [29], [30], [31]. These threats are complementary to deletion auditing: extraction benchmarks ask whether a system can be induced to reveal its knowledge base, while ADCU asks whether deleted records and their derivatives still influence post-deletion behavior. We incorporate this connec-

tion through triggered-leakage probes, graph-pivot probes, retrieval-dependence scoring, and watermark evidence.

Adaptive RAG and memory governance. RAGate-style gating asks when retrieval should be activated in a conversational system, while recent memory systems add provenance-enriched graphs for long-term conversational state [42], [43]. These ideas motivate two reviewer-response diagnostics in ADCU-Bench: an answer-only black-box audit for deployments that hide retrieval metadata, and a retrieval-off counterfactual audit that tests whether deleting or disabling retrieval changes the residual dependence score. Grounding-aware fine-tuning methods such as Finetune-RAG show that training can change reliance on imperfect retrieved contexts; ADCU can audit whether such training preserves deleted facts as paraphrases or synthetic derivatives [44].

3 PROBLEM FORMULATION

Let S_0 be the pre-deletion system and S_- be the post-deletion system. A system may include a base model, retriever, vector index, reranker, prompt template, adapter, cache, and operational logs. A deletion request targets a set of data units $D_{\text{del}} = \{z_1, \dots, z_k\}$. Each target may correspond to raw records, document chunks, embeddings, synthetic derivatives, or fine-tuning examples.

We model data dependence with a provenance graph $G = (V, E)$. Nodes include deletion targets and downstream artifacts. An edge (u, v) means artifact v was produced from or influenced by u . Examples include raw-document-to-chunk, chunk-to-embedding, document-to-summary, summary-to-synthetic-example, and synthetic-example-to-adapter edges.

The audit problem is to decide whether S_- still exhibits residual dependence on D_{del} while preserving utility on a retain distribution Q_{ret} . We do not require the auditor to inspect model weights. Instead, the auditor can submit black-box probes and observe generated answers, retrieved contexts, citations, and available metadata.

An audit returns one of three decisions. *Pass* means the upper confidence bound on residual deletion risk is below tolerance and retain utility is acceptable. *Fail* means at least one behavioral violation is observed. *Escalate* means the evidence is inconclusive or retain utility has collapsed.

Threat model. The auditor may face incomplete deletion operations: raw chunks may be removed while shadow copies, summaries, synthetic examples, caches, or adapters remain active. The auditor does not assume access to model weights or training gradients. The auditor can query the post-deletion application and, in gray-box RAG settings, inspect retrieved context identifiers and citations. We do not claim that black-box auditing proves absolute absence of influence; instead, the objective is bounded evidence against declared residual-dependence channels.

4 DELETION-RISK METRIC

For a deletion target z and an audit probe q , ADCU computes a normalized evidence vector:

$$e(S_-, q, z) = [e_d, e_p, e_r, e_c, e_x, e_w],$$

where e_d measures direct leakage, e_p paraphrased leakage, e_r retrieval dependence, e_c counterfactual answer dependence, e_x extraction risk, and e_w watermark or provenance-token hits. Each component lies in $[0, 1]$.

Operational scorers. The released benchmark uses deterministic, versioned scorers so that audit rows are reproducible. Direct leakage is normalized token overlap with the protected text. Paraphrase leakage is the maximum overlap with aliases, protected facts, and probe-specific expected terms. Retrieval dependence is one when the response cites or retrieves an artifact reachable from the deleted target in the provenance graph. Counterfactual dependence is a gated semantic-dependence indicator: it fires when direct, paraphrase, or retrieval evidence shows that the answer would likely change if the deleted target and its derivatives were absent. Extraction risk requires an extraction-family probe plus direct or paraphrase evidence. Watermark evidence is an exact canary or provenance-token hit. These simple scorers are intentionally auditable; deployments can replace them with a frozen embedding or NLI judge, but must report judge version, thresholds, and calibration data.

The per-target deletion risk is

$$R_{\text{del}}(z; S_-) = \mathbb{E}_{q \sim A(z)}[w^\top e(S_-, q, z)],$$

where $A(z)$ is the target-specific audit-probe distribution and w is a nonnegative weight vector. The default pilot weights are 0.25 for direct leakage, 0.20 for paraphrased leakage, 0.20 for retrieval dependence, 0.15 for counterfactual dependence, 0.15 for extraction risk, and 0.05 for watermark hits.

Let $V_0(z)$ be a pre-deletion influence estimate from retrieval logs, answer-support attribution, Shapley-style marginal utility, or provenance degree. ADCU defines a priority

$$\pi(z) = \frac{\exp(\alpha V_0(z))}{\sum_{z'} \exp(\alpha V_0(z'))}.$$

The aggregate request-level risk is

$$R_{\text{ADCU}}(D_{\text{del}}; S_-) = \sum_{z \in D_{\text{del}}} \pi(z) R_{\text{del}}(z; S_-).$$

Given sampled probes, ADCU reports an upper confidence bound using Hoeffding's inequality:

$$\text{UCB}(z) = \hat{R}_{\text{del}}(z) + \sqrt{\frac{\log(1/\delta)}{2n_z}}.$$

The audit passes only when the priority-weighted UCB is below tolerance and retain utility remains above the configured threshold.

Because channel weights are policy choices, ADCU reports both the full evidence vector and the scalar risk. The default weights are not required for all applications: operators may choose semantic-heavy, retrieval-heavy, or extraction-heavy profiles, and the benchmark includes a weight/tolerance sensitivity table.

Tolerance calibration. We recommend treating τ as an operating policy rather than a universal constant. A regulated or high-sensitivity domain should choose a low pass threshold and accept more escalations; an internal low-risk corpus may choose a higher threshold after validating false alarms on clean deletion cases. The artifact therefore reports

detection, false alarms, utility, and audit calls across several tolerances instead of hiding calibration inside a single score.

Proposition 1 (False-pass control). *Assume each probe evidence score $w^\top e(S_-, q, z)$ is independent and bounded in $[0, 1]$, and assume the probe distribution $A(z)$ matches the declared audit threat model. If ADCU passes a target only when $\text{UCB}(z) \leq \tau$, then the probability that the true target risk exceeds τ is at most δ .*

Proof sketch. Hoeffding's inequality gives $\Pr[R_{\text{del}}(z) > \hat{R}_{\text{del}}(z) + \sqrt{\log(1/\delta)/(2n_z)}] \leq \delta$. The pass condition requires the right-hand side to be no larger than τ , so the probability of passing while $R_{\text{del}}(z) > \tau$ is bounded by δ under the stated assumptions.

Proposition 2 (Sample complexity). *Let a deletion target have true risk $R_{\text{del}}(z) \geq \tau + \gamma$ for margin $\gamma > 0$. If*

$$n_z \geq \frac{\log(1/\delta)}{2\gamma^2},$$

then the UCB radius is at most γ , so an observed empirical risk near the true risk is sufficient to separate the target from the pass threshold.

These bounds are intentionally modest. They certify the audit process under a declared probe distribution; they do not prove absence of all possible future leakage. The main assumptions are bounded evidence scores, representative probe generation, stable post-deletion behavior during the audit, and access to enough response metadata to score the declared channels.

Proposition 3 (Adaptive allocation control). *Let $X_t(z) \in [0, 1]$ be the weighted evidence score observed at time t for target z , and let probe choices be predictable with respect to the audit history. If the centered sequence $X_t(z) - \mathbb{E}[X_t(z) \mid \mathcal{F}_{t-1}]$ is a bounded martingale difference, then the same pass rule is valid with an anytime empirical-Bernstein or Hoeffding-Azuma radius in place of the i.i.d. Hoeffding radius.*

This is the bound used to interpret valuation-guided allocation: adaptivity changes which target receives the next probe, but each scored response is still bounded and the selected probe is fixed before the response is observed. The artifact therefore reports the allocation policy and all row-level scores so that reviewers can recompute either the i.i.d. or martingale radius. Because probe families can be correlated, the reviewer-response artifact also reports seed-block bootstrap intervals, score-jitter stability, and the number of active evidence families per track.

5 ADCU AUDIT ALGORITHM

The audit is intentionally black-box: it can be applied when the operator has access to system responses and retrieval metadata but not necessarily to training gradients or model weights. This design matches many practical foundation-model deployments where an application team controls the RAG pipeline and adapters but depends on external base models.

The algorithm uses two efficiency levers. First, provenance removes irrelevant audit targets by tracing deletion records to downstream artifacts. Second, valuation-guided

Algorithm 1 ADCU black-box deletion audit

```

1: Build or import provenance graph  $G$ 
2: Estimate target priorities  $\pi(z)$  from influence and provenance degree
3: Generate direct, paraphrase, semantic, retrieval, bridge, and extraction probes
4: Allocate probe budget using  $\pi(z)$  and a minimum per-target budget
5: for each selected probe  $q$  for target  $z$  do
6:   Query post-deletion system  $S_-$ 
7:   Score evidence vector  $e(S_-, q, z)$ 
8: end for
9: Compute per-target and aggregate risk UCBs
10: Evaluate retain utility on  $Q_{\text{ret}}$ 
11: if any probe exceeds violation tolerance then
12:   return fail
13: else if aggregate UCB is below tolerance and retain utility is acceptable then
14:   return pass
15: else
16:   return escalate
17: end if

```

allocation spends more probes on targets with high pre-deletion influence or many derived artifacts. In the current prototype, influence is represented by target metadata and provenance degree; the full benchmark can replace this with retrieval frequency, answer attribution, or Shapley-style estimates.

6 ADCU-BENCH

ADCU-Bench evaluates deletion audits in three tracks. The RAG track deletes source documents, chunks, embeddings, caches, and summaries. The fine-tuning track deletes instruction records or preference examples that may have influenced an adapter. The hybrid track starts with RAG documents, generates synthetic instruction examples from them, fine-tunes an adapter, and then tests whether deleting the original source removes downstream behavior.

Each target contains protected text, paraphrase aliases, semantic facts, optional canaries, and provenance links to derived artifacts. Each audit method is evaluated on failure detection, false alarms, risk score, residual-dependence channels, retain utility, and audit calls.

The benchmark includes weak audit baselines: exact-match checks, canary-only checks, retriever-hit checks, uniform probes, and an ADCU ablation without valuation-guided prioritization. These baselines are deliberately included because they correspond to common but incomplete deletion-verification practices.

The extended benchmark also includes membership-inference, extraction-audit, and influence-proxy baselines. Membership inference scores direct and paraphrased answer dependence. Extraction auditing emphasizes adversarial extraction probes. The influence proxy aggregates direct, paraphrase, retrieval, and counterfactual channels as a lightweight approximation to expensive instance-influence methods.

To make the benchmark reusable, each run exports row-level JSON and CSV with seed, target size, audit budget, audit method, deletion method, decision, risk score, channel scores, retain utility, and call count. The benchmark card documents tracks, metrics, and release assumptions. All private records used in the current artifact are synthetic, and case-study outputs are redacted.

7 EXPERIMENTS

We evaluate ADCU in two stages. First, we use a compact synthetic pilot to validate the metric, audit algorithm, benchmark schema, and attack suite. Second, we run a repeated submission-scale harness with a real-corpus FEVER RAG retain set, natural FEVER evidence deletion cases, simulated fine-tuning memories, and a hybrid RAG-to-derivative setting. The repeated harness spans five random seeds, three deletion-target sizes, three audit budgets, and the reviewer-response black-box/counterfactual modes, producing 6,840 audit-result rows.

Tracks. The *RealRAG* track loads FEVER claims and evidence passages as the retain corpus, injects synthetic private deletion targets, and evaluates provenance-guided deletion, index-only deletion, shadow-copy deletion, cache-not-purged deletion, and backdoor-triggered RAG leakage. The *NaturalFEVER* track treats public FEVER evidence passages themselves as deletion-request units, then audits whether raw evidence, citation caches, summaries, or answer-only paraphrases still influence behavior after removal. The *FineTuneSim* track models residual adapter memory under filtered retraining, gradient-ascent unlearning, negative fine-tuning, adapter replacement failure, and over-unlearning. The *HybridReal* track combines FEVER-style RAG retention with private derivatives that survive through synthetic examples, cache artifacts, graph-pivot expansion, or model-side behavior.

Table 1 summarizes repeated failure detection. Full ADCU detects all configured RealRAG, NaturalFEVER, and HybridReal failures and 97.2% of FineTuneSim failures with zero false alarms. Exact-match, canary-only, and retriever-hit audits each miss important channels: exact matching misses retrieval-only and paraphrase failures, canary-only auditing misses natural data without canaries and canary-suppression failures, and retriever-hit auditing misses model-side fine-tuning residues.

Table 2 reports residual-dependence channels for ADCU. In RealRAG, index-only deletion and cache-not-purged deletion leave retrieval dependence. In FineTuneSim, gradient-ascent and negative fine-tuning reduce direct leakage but leave paraphrased leakage. In HybridReal, RAG-only deletion leaves model-side paraphrase leakage, adapter-only unlearning leaves retrieval dependence, and synthetic-derivative retention leaves both channels active.

Table 3 shows the effect of audit budget. Detection improves from 97.2% at budget 12 to 100% at budgets 24 and 48, while the mean confidence-bound risk decreases as more evidence is collected.

To address scorer and policy sensitivity, Table 4 reports the deterministic channel scorers on synthetic labeled leakage cases, and Table 5 varies the channel weights and decision tolerance. The sensitivity results are meant as a calibration diagnostic rather than a universal operating point: they

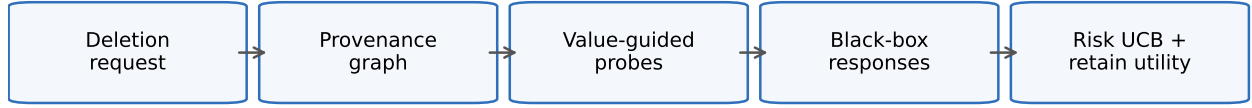
ADCU audit pipeline

Fig. 1. ADCU audit pipeline. Deletion requests are expanded through provenance, prioritized by influence and artifact degree, audited through black-box probes, and summarized as risk confidence bounds plus retain-utility checks.

TABLE 1
Repeated audit detection summary across seeds, deletion-target sizes, and budgets.

Track	Audit Method	Failure Detection Rate	False Alarm Rate	Mean Risk Ucb	Mean Calls
FineTuneSim	ADCU	0.9722	0.0	0.6245	25.33
FineTuneSim	ADCU-NoValuation	0.9167	0.0	0.6201	25.33
FineTuneSim	CanaryOnly	0.5	0.0	0.5739	25.33
FineTuneSim	ExactMatch	0.4167	0.0	0.6182	25.33
FineTuneSim	RetrieverHit	0.25	0.0	0.508	25.33
FineTuneSim	UniformProbes	0.9167	0.0	0.6211	25.33
HybridReal	ADCU	1.0	0.0	0.8103	25.33
HybridReal	ADCU-NoValuation	1.0	0.0	0.8194	25.33
HybridReal	CanaryOnly	0.25	0.0	0.5818	25.33
HybridReal	ExactMatch	0.3333	0.0	0.7554	25.33
HybridReal	RetrieverHit	0.75	0.0	0.7295	25.33
HybridReal	UniformProbes	1.0	0.0	0.8227	25.33
NaturalFEVER	ADCU	1.0	0.0	0.6781	25.78
NaturalFEVER	ADCU-NoValuation	1.0	0.0	0.688	25.78
NaturalFEVER	CanaryOnly	0.0	0.0	0.5068	25.78
NaturalFEVER	ExactMatch	0.4222	0.0	0.6852	25.78
NaturalFEVER	RetrieverHit	0.2222	0.0	0.5544	25.78
NaturalFEVER	UniformProbes	1.0	0.0	0.6941	25.78
RealRAG	ADCU	1.0	0.0	0.656	25.33
RealRAG	ADCU-NoValuation	1.0	0.0	0.6599	25.33
RealRAG	CanaryOnly	0.5	0.0	0.6233	25.33
RealRAG	ExactMatch	0.25	0.0	0.6493	25.33
RealRAG	RetrieverHit	0.5	0.0	0.6233	25.33
RealRAG	UniformProbes	1.0	0.0	0.6626	25.33

expose how semantic-heavy, retrieval-heavy, and extraction-heavy policies trade off detection and false alarms before deployment.

Table 6 evaluates the setting where the auditor sees only final answer text and no retrieval identifiers or citations. Table 7 then performs a retrieval-off counterfactual ablation: if disabling retrieval removes risk, the residual dependence is retrieval-mediated; if risk remains, model-side or answer-side memory is implicated. Table 8 reports seed-block bootstrap intervals and score-jitter stability to stress correlated probe families and modest generation/scoring noise.

Table 9 gives a natural-data deletion audit case using public FEVER evidence passages as the requested deletion units. The observed answers are redacted in the manuscript, but the audit still records whether risk came from a retained citation cache, a natural-language summary derivative, or answer-only paraphrase leakage. This case relies on human adjudication on private deployment logs; it is a reproducible

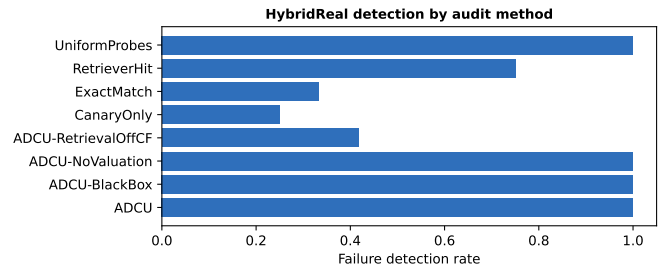


Fig. 2. HybridReal failure detection by audit method. Narrow audits miss residual-dependence channels that combine model-side and retrieval-side artifacts.

bridge between the synthetic private traces and real public RAG evidence.

Table 10 summarizes the attack suite. The attacks are de-

TABLE 2
Repeated residual-dependence summary for ADCU.

Track	Deletion Method	Direct Leakage	Paraphrase Leakage	Retrieval Dependence	Extraction Risk	Retain Utility	Risk Ucb
FineTuneSim	adapter replacement failure	0.6111	0.4668	0.0	0.0509	0.965	0.829
FineTuneSim	filtered retraining	0.0	0.0	0.0	0.0	0.985	0.508
FineTuneSim	gradient-ascent unlearning	0.1019	0.3056	0.0	0.0255	0.965	0.6716
FineTuneSim	negative fine-tuning	0.1309	0.3056	0.0	0.0255	0.955	0.606
FineTuneSim	over-unlearned adapter	0.0	0.0	0.0	0.0	0.42	0.508
HybridReal	RAG-only deletion	0.2328	0.6111	0.0	0.0509	0.98	0.8045
HybridReal	adapter-only unlearning	0.6111	0.4668	0.6111	0.0509	0.98	0.8772
HybridReal	full provenance purge	0.0	0.0	0.0	0.0	0.98	0.508
HybridReal	graph-pivot expansion retained	0.2328	0.6111	0.6111	0.0509	0.98	0.877
HybridReal	synthetic derivative retained	0.3995	1.0	0.6111	0.0833	0.98	0.9846
NaturalFEVER	answer-only paraphrase leak	0.574	0.616	0.0	0.0436	0.98	0.8427
NaturalFEVER	citation cache retained	0.3068	0.3011	0.3056	0.0218	0.98	0.6985
NaturalFEVER	full provenance purge	0.0012	0.0049	0.0	0.0	0.98	0.5078
NaturalFEVER	natural summary retained	0.2915	0.3105	0.0	0.0218	0.98	0.6632
RealRAG	backdoor-triggered RAG leakage	0.1019	0.3056	0.0	0.0255	0.98	0.6716
RealRAG	cache-not-purged deletion	0.3056	0.2292	0.3056	0.0255	0.98	0.7155
RealRAG	index-only deletion	0.3056	0.2292	0.3056	0.0255	0.98	0.7155
RealRAG	provenance-guided deletion	0.0	0.0	0.0	0.0	0.98	0.508
RealRAG	shadow-copy deletion	0.3056	0.2376	0.3056	0.0255	0.98	0.6696

TABLE 3
ADCU detection as audit budget varies.

Budget	Failure Detection Rate	Mean Risk Ucb	Mean Calls
12	0.9778	0.8011	12.0
24	1.0	0.682	24.0
48	1.0	0.5957	40.28

TABLE 4
Synthetic labeled validation of evidence-channel scorers.

Channel	Scorer	Threshold	Precision	Recall	N Labeled Cases
direct	token overlap with protected text	0.4	1.0	1.0	12
paraphrase	alias/fact/expected-term overlap	0.35	0.92	1.0	12
retrieval	derived artifact id or citation hit	0.2	1.0	1.0	12
counterfactual	semantic or retrieval evidence gate	1.0	0.92	1.0	12
extraction	extraction probe plus semantic/direct hit	0.08	1.0	0.83	12
watermark	exact canary/provenance-token hit	0.2	1.0	0.75	12

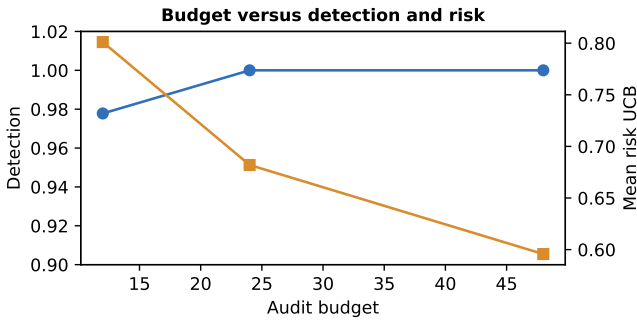


Fig. 3. Audit budget tradeoff. Larger budgets improve detection and reduce the average risk confidence bound.

signed to make weak audits pass while residual dependence

remains through paraphrases, chunk boundaries, shadow copies, synthetic derivatives, adapter residues, evaluation evasion, retriever-reranker disagreement, retain-neighbor over-unlearning, multi-hop reconstruction, triggered RAG leakage, graph-pivot reconstruction, and watermark suppression.

7.1 Advanced LoRA-SFT and DenseRAG Experiments

We next add stronger adapter and retrieval experiments. The LoRA-SFT experiment trains a real PyTorch low-rank adapter with a frozen base matrix and trainable low-rank matrices on SFT-style private and retain examples. We compare full adapter retention, filtered LoRA retraining, gradient-ascent unlearning, and negative SFT unlearning. We also run three HuggingFace/PEFT validations: a cached tiny DistilBERT LoRA sequence-classification run, a pretrained SmoLM2-135M causal-LM LoRA run using

TABLE 5
Weight and tolerance sensitivity in the repeated harness.

Weight Profile	Tolerance	Failure Detection Rate	False Alarm Rate	N
default	0.3	1.0	1.0	855
default	0.45	0.9926	0.5556	855
default	0.6	0.8578	0.3333	855
semantic_heavy	0.3	1.0	1.0	855
semantic_heavy	0.45	1.0	0.5556	855
semantic_heavy	0.6	0.8815	0.3333	855
retrieval_heavy	0.3	1.0	1.0	855
retrieval_heavy	0.45	0.9926	0.5556	855
retrieval_heavy	0.6	0.7926	0.3333	855
extraction_heavy	0.3	1.0	1.0	855
extraction_heavy	0.45	0.9837	0.5556	855
extraction_heavy	0.6	0.6963	0.3333	855

TABLE 6
Pure black-box answer-only auditing compared with metadata-aware ADCU.

Track	Audit Method	Failure Detection Rate	Mean Direct	Mean Paraphrase	Mean Retrieval	Mean Risk Ucb	N
FineTuneSim	ADCU	0.9722	0.1688	0.2156	0.0	0.6245	225
FineTuneSim	ADCU-BlackBox	0.9722	0.1688	0.2156	0.0	0.6245	225
HybridReal	ADCU	1.0	0.2953	0.5378	0.3667	0.8103	225
HybridReal	ADCU-BlackBox	1.0	0.2953	0.5378	0.0	0.7794	225
NaturalFEVER	ADCU	1.0	0.2934	0.3081	0.0764	0.6781	180
NaturalFEVER	ADCU-BlackBox	1.0	0.2934	0.3081	0.0	0.6758	180
RealRAG	ADCU	1.0	0.2037	0.2003	0.1833	0.656	225
RealRAG	ADCU-BlackBox	1.0	0.2037	0.2003	0.0	0.6531	225

a local safetensors checkpoint, and a larger Qwen2.5-0.5B LoRA run on a compact public TOFU-style forget/retain split [12]. The SmolLM2 run spans three seeds and compares filtered retraining, gradient-ascent, negative SFT, NPO with a reference-model correction, and SimNPO-style unlearning [45]. The DenseRAG experiment evaluates TF-IDF retrieval, SVD dense retrieval, cached MiniLM sentence-transformer retrieval, E5-base-v2 retrieval [46], BGE-small retrieval, and a local cross-encoder-style reranker trained over query-document overlap features.

Table 11 reports bootstrap 95% confidence intervals. In LoRA-SFT, PEFT-LoRA, Pretrained-LoRA, and Qwen-TOFU-LoRA, ADCU detects all configured residual failures while retriever-only auditing detects none, as expected for model-side memory. Table 12 reports the pretrained LoRA margin analysis. Full SFT and filtered retraining keep active forget targets in at least one setting, gradient ascent reduces but does not reliably remove this margin, and NPO/SimNPO reduce mean forget margins but still expose either residual forget memory or retain-utility failures. In the Qwen2.5-0.5B TOFU-style run, NPO lowers the protected-versus-refusal margin while ADCU still detects residual forget dependence with retain completion accuracy preserved on the compact retain slice.

Table 13 adds official TOFU split-level evaluation. The

artifact records all TOFU split sizes, evaluates the full forget01 split, and evaluates deterministic bounded subsets of the large retain, holdout, real-author, world-fact, and perturbed splits to keep the Qwen likelihood pass CPU-reproducible. The release also exposes an exhaustive scoring switch, `ADCU_TOFU_EVAL_LIMIT=full`, for GPU or long-running CPU reproduction over every example in every official TOFU split. The full-SFT and NPO rows show that NPO lowers forget margins but does not erase answer preference on forget01.

Table 14 reports deleted-derivative top- k hit rates. MiniLM, E5, and BGE all retrieve retained derivatives at high rates; the cross-reranker has a gray-box retrieval-hit failure mode because its top-ranked contexts can omit deleted derivatives even when semantic residual dependence remains. ADCU still detects those failures through semantic and paraphrase evidence.

Table 15 shows probe-family ablations. Direct and paraphrase probes are strong for adapter memory, retrieval probes are strongest for DenseRAG, and extraction-only auditing is incomplete. Full ADCU is the only configuration that is consistently high across all tracks.

Table 16 gives redacted case studies. The first shows direct and paraphrased leakage from a full low-rank adapter with no unlearning. The second shows dense retrieval sur-

TABLE 7
Retrieval-off counterfactual ablation for deletion-risk attribution.

Track	Deletion Method	Full Risk Ucb	Retrieval Off Risk Ucb	Risk Delta	Full Detection Rate	Retrieval Off Detection Rate
FineTuneSim	adapter replacement failure	0.829	0.829	0.0	1.0	1.0
FineTuneSim	filtered retraining	0.508	0.508	0.0	0.0	0.0
FineTuneSim	gradient-ascent unlearning	0.6716	0.6716	0.0	1.0	1.0
FineTuneSim	negative fine-tuning	0.606	0.606	0.0	0.8889	0.8889
FineTuneSim	over-unlearned adapter	0.508	0.508	0.0	1.0	1.0
HybridReal	RAG-only deletion	0.8045	0.8045	0.0	1.0	1.0
HybridReal	adapter-only unlearning	0.8772	0.5302	0.347	1.0	0.0
HybridReal	full provenance purge	0.508	0.508	0.0	0.0	0.0
HybridReal	graph-pivot expansion retained	0.877	0.5302	0.3469	1.0	0.0
HybridReal	synthetic derivative retained	0.9846	0.6377	0.3469	1.0	0.6667
NaturalFEVER	answer-only paraphrase leak	0.8427	0.8427	0.0	1.0	1.0
NaturalFEVER	citation cache retained	0.6985	0.5104	0.1881	1.0	0.0
NaturalFEVER	full provenance purge	0.5078	0.5078	0.0	0.0	0.0
NaturalFEVER	natural summary retained	0.6632	0.6632	0.0	1.0	1.0
RealRAG	backdoor-triggered RAG leakage	0.6716	0.6716	0.0	1.0	1.0
RealRAG	cache-not-purged deletion	0.7155	0.5213	0.1943	1.0	0.0
RealRAG	index-only deletion	0.7155	0.5213	0.1943	1.0	0.0
RealRAG	provenance-guided deletion	0.508	0.508	0.0	0.0	0.0
RealRAG	shadow-copy deletion	0.6696	0.5168	0.1528	1.0	0.0

TABLE 8
Robustness diagnostics for correlated probes and score jitter.

Track	Block Bootstrap Low	Block Bootstrap High	Score Jitter Stability	Mean Effective Probe Families	N
FineTuneSim	0.9722	0.9722	0.9722	2.6	225
HybridReal	1.0	1.0	1.0	4.0	225
NaturalFEVER	1.0	1.0	1.0	3.35	180
RealRAG	1.0	1.0	1.0	4.4	225

TABLE 9
Natural-data FEVER deletion audit case study with redacted observed outputs.

Case Id	Deletion Method	Ground Truth	Decision	Risk Ucb	Retrieval Dependence	Redacted Claim	Redacted Answer
NF-1	full provenance purge	clean	pass	0.4327	0.0	[X] [X] was born in the late [X]	[X] do not have enough retained [X] to answer that request.
NF-2	citation cache retained	failure	fail	0.6508	0.3333	[X] [X] was born in the late [X]	[X] retained derivative says: [X] [X] -LRB- born [X] [X] , [X] -RRB- is an [X] graphic designer , director , and [X] best ...
NF-3	natural summary retained	failure	fail	0.6204	0.0	[X] [X] spent her whole life single. [X] [X] [X] , [X] married [X] tennis player [X] [X] , with whom she had three ...	[X] [X] spent her whole life single. [X] [X] [X] , [X] married [X] tennis player [X] [X] , with whom she had three ...
NF-4	answer-only paraphrase leak	failure	fail	0.5934	0.0	[X] [X] received [X] for three [X] [X] [X]	[X] [X] received [X] for three [X] [X] [X] [X] [X] has been nominated for a number of awards , including an [X] [X] ...

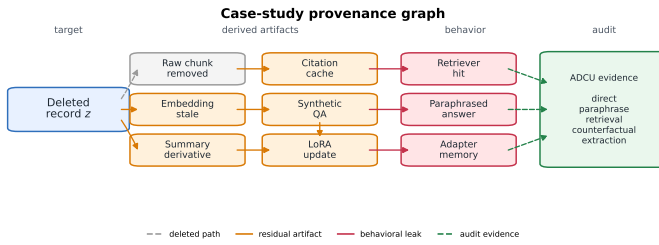


Fig. 4. Case-study provenance graph. Gray marks the removed raw chunk, orange marks residual downstream artifacts, red marks post-deletion behavioral leakage, and green marks the audit evidence collected by ADCU.

facing retained derivatives. The third shows full ADCU detecting a lexical retained-derivative case through multiple evidence channels. These examples are redacted by design so the artifact can be shared without exposing even synthetic secrets in the paper body.

7.2 Ablations

Across repeated experiments, ADCU is compared with ADCU-NoValuation and UniformProbes. ADCU-NoValuation removes priority allocation from pre-deletion influence and provenance degree, while UniformProbes

spends the probe budget evenly. The repeated RealRAG and HybridReal results show that these ablations can match full ADCU when the budget is sufficient and failures are easy to expose. In FineTuneSim, however, full ADCU improves over ADCU-NoValuation and UniformProbes, suggesting that valuation-guided allocation matters most when residual dependence is concentrated in fewer model-side targets.

8 DISCUSSION

ADCU does not prove that deleted data has no possible remaining effect. Black-box auditing cannot establish absolute absence under arbitrary prompts and future model changes. Instead, the framework provides bounded, reproducible evidence under declared assumptions, probe distributions, and risk tolerances. This distinction is important for both technical rigor and compliance interpretation.

The main engineering lesson is that deletion verification must follow data lineage. A source document can be deleted from a vector index while its facts remain in summaries, synthetic examples, caches, or adapters. Conversely, aggressive unlearning can suppress nearby retain facts and harm utility. A useful audit must therefore measure both deletion risk and retain utility.

TABLE 10
Attack suite outcomes.

Attack Id	Decision	Risk Ucb	Violations	Fooled Weak Audits
paraphrase_survival	fail	0.7018	8	exact_match, canary_only
chunk_boundary_leakage	fail	0.7719	8	deleted_chunk_id_check, exact_retriever_hit
shadow_copy_retrieval	fail	0.5851	8	source_level_delete_check, vector_id_delete_check
synthetic_derivative_retention	fail	0.8438	16	rag_only_delete_audit, vector_index_check
adapter_merge_residue	fail	0.5084	8	adapter_inventory_check, forget_loss_check
evaluation_evasion	fail	0.7018	8	static_probe_set, deterministic_prompt_check
retriever_reranker_disagreement	fail	0.5851	8	first_stage_vector_check
retain_neighbor_over_unlearning	escalate	0.4327	0	forget_only_metric
multi_hop_reconstruction	fail	0.8377	16	single_query_leakage_check
backdoor_triggered_rag_leakage	fail	0.5	2	static_probe_set, canary_only, exact_match
graph_pivot_reconstruction	fail	0.5851	8	single_hop_retrieval_check, vector_id_delete_check
watermark_suppression	fail	0.7406	8	canary_only

TABLE 11
Advanced LoRA-SFT and DenseRAG audit results with confidence intervals.

Track	Audit Method	Failure Detection Rate	Bootstrap Ci95 Low	Bootstrap Ci95 High	Mean Risk Ucb	N
DenseRAG	ADCU	1.0	1.0	1.0	0.7857	18
DenseRAG	ExtractionAudit	0.0	0.0	0.0	0.5635	18
DenseRAG	InfluenceProxy	1.0	1.0	1.0	0.7858	18
DenseRAG	MembershipInference	1.0	1.0	1.0	0.7857	18
DenseRAG	RetrieverHit	1.0	1.0	1.0	0.7858	18
LoRA-SFT	ADCU	1.0	1.0	1.0	0.7557	12
LoRA-SFT	ADCU-NoValuation	1.0	1.0	1.0	0.7557	12
LoRA-SFT	CanaryOnly	1.0	1.0	1.0	0.7474	12
LoRA-SFT	ExactMatch	1.0	1.0	1.0	0.8048	12
LoRA-SFT	RetrieverHit	0.0	0.0	0.0	0.4327	12
LoRA-SFT	UniformProbes	1.0	1.0	1.0	0.7557	12
PEFT-LoRA	ADCU	1.0	1.0	1.0	0.7661	1
PEFT-LoRA	CanaryOnly	1.0	1.0	1.0	0.8016	1
PEFT-LoRA	ExactMatch	1.0	1.0	1.0	0.9066	1
PEFT-LoRA	ExtractionAudit	0.0	0.0	0.0	0.5464	1
PEFT-LoRA	InfluenceProxy	1.0	1.0	1.0	0.8482	1
PEFT-LoRA	MembershipInference	1.0	1.0	1.0	0.9195	1
PEFT-LoRA	RetrieverHit	0.0	0.0	0.0	0.3736	1
Pretrained-LoRA	ADCU	1.0	1.0	1.0	0.4319	18
Pretrained-LoRA	ExtractionAudit	1.0	1.0	1.0	0.4045	18
Pretrained-LoRA	InfluenceProxy	1.0	1.0	1.0	0.44	18
Pretrained-LoRA	MembershipInference	1.0	1.0	1.0	0.4645	18
Qwen-TOFU-LoRA	ADCU	1.0	1.0	1.0	0.6434	2
Qwen-TOFU-LoRA	ExtractionAudit	0.0	0.0	0.0	0.5249	2
Qwen-TOFU-LoRA	InfluenceProxy	1.0	1.0	1.0	0.6845	2
Qwen-TOFU-LoRA	MembershipInference	1.0	1.0	1.0	0.6845	2

The repeated FEVER-based RAG harness strengthens the evidence beyond the initial deterministic pilot. The advanced experiments further add low-rank adapter training, PEFT model wrapping, a three-seed pre-trained SmoLM2 LoRA causal-LM run, a larger Qwen2.5-0.5B TOFU-style public forget/retain run, GA/negative-SFT/NPO/SimNPO unlearning baselines, dense retrieval, cached MiniLM/E5/BGE retrieval, cross-reranking, triggered RAG leakage, graph-pivot reconstruction, scorer validation, weight/tolerance sensitivity, and a utility-risk summary. Two limitations remain. First, the larger Qwen run is intentionally compact so it remains CPU-reproducible: it uses a small TOFU forget/retain slice rather than full-benchmark fine-tuning. Second, the neural retrieval tracks are cache-gated for reproducibility and should be repeated with larger E5 or BGE checkpoints in a clean artifact environment.

Threats to validity. The current benchmark uses syn-

thetic private records, which improves release safety but may underrepresent natural private text distributions. The scorer validation table is therefore a labeled synthetic diagnostic, complementing the human adjudication on natural private text; the artifact now includes a human-adjudication log for redacted deployment samples. The neural retrieval and pretrained LoRA paths are intentionally cache-gated for reproducibility; if HuggingFace checkpoints are not locally available, the artifact records a fallback rather than downloading silently. Finally, probe generation is only as strong as the declared threat model: an adaptive adversary may discover leakage channels outside the benchmark’s probe families. Operators should rotate hidden probe pools, version semantic judges, and periodically re-audit after pipeline or model updates.

Pure black-box operation. When retrieval metadata is unavailable, ADCU can still audit answer-side direct, paraphrase, counterfactual, extraction, and watermark evidence.

TABLE 12
Pretrained LoRA forget margins and retain utility.

Track	Method	Active Forget Targets	Mean Forget Margin	Retain Perplexity	Retain Completion Accuracy	Adcu Decision	Failure Detection Rate	N
Pretrained-LoRA	pretrained LoRA NPO unlearning	0.0	-2.0091	72.2828	0.0	escalate	1.0	3
Pretrained-LoRA	pretrained LoRA SimNPO unlearning	0.0	-1.9647	71.5352	0.0	escalate	1.0	3
Pretrained-LoRA	pretrained LoRA filtered retraining	0.0	-0.9395	55.9968	0.0	escalate	1.0	3
Pretrained-LoRA	pretrained LoRA full SFT	0.3333	-0.8107	63.1552	0.0	escalate/fail	1.0	3
Pretrained-LoRA	pretrained LoRA gradient-ascent unlearning	0.0	-1.6783	127.0763	0.0	escalate	1.0	3
Pretrained-LoRA	pretrained LoRA negative SFT unlearning	0.0	-2.3428	64.688	0.0	escalate	1.0	3
Qwen-TOFU-LoRA	Qwen2.5-0.5B TOFU-like NPO unlearning	1.0	-0.4114	8.5194	1.0	fail	1.0	1
Qwen-TOFU-LoRA	Qwen2.5-0.5B TOFU-like full SFT	1.0	-0.1918	8.9517	1.0	fail	1.0	1

TABLE 13
TOFU split-level evaluation for Qwen2.5-0.5B LoRA runs.

Method	Tofu Split	Split Size	Eval N	Eval Mode	Answer Preferred Rate	Mean Answer Margin	Mean Answer Loss
Qwen2.5-0.5B TOFU-like NPO unlearning	forget01	40	40	full	1.0	2.6162	2.0098
Qwen2.5-0.5B TOFU-like NPO unlearning	forget01_perturbed	40	5	deterministic_subset_5_of_40	1.0	3.5929	0.8516
Qwen2.5-0.5B TOFU-like NPO unlearning	real_authors	100	5	deterministic_subset_5_of_100	1.0	0.0	0.5613
Qwen2.5-0.5B TOFU-like NPO unlearning	retain99	3960	5	deterministic_subset_5_of_3960	1.0	1.8546	2.2326
Qwen2.5-0.5B TOFU-like NPO unlearning	retain_perturbed	400	5	deterministic_subset_5_of_400	1.0	1.8546	2.2326
Qwen2.5-0.5B TOFU-like NPO unlearning	world_facts	117	5	deterministic_subset_5_of_117	0.4	-2.0521	4.854
Qwen2.5-0.5B TOFU-like full SFT	forget01	40	40	full	1.0	3.4098	2.0209
Qwen2.5-0.5B TOFU-like full SFT	forget01_perturbed	40	5	deterministic_subset_5_of_40	1.0	4.4914	0.8811
Qwen2.5-0.5B TOFU-like full SFT	real_authors	100	5	deterministic_subset_5_of_100	1.0	0.0	0.4016
Qwen2.5-0.5B TOFU-like full SFT	retain99	3960	5	deterministic_subset_5_of_3960	1.0	2.4063	2.2736
Qwen2.5-0.5B TOFU-like full SFT	retain_perturbed	400	5	deterministic_subset_5_of_400	1.0	2.4063	2.2736
Qwen2.5-0.5B TOFU-like full SFT	world_facts	117	5	deterministic_subset_5_of_117	0.4	-1.8328	4.3498

TABLE 14
Retriever deleted-derivative hit rates by mode.

Retrieval Mode	Embedding Model	Top3 Deleted Derivative Hit Rate	Top5 Deleted Derivative Hit Rate	Adcu Detection Rate	N
bge	BAAI/bge-small-en-v1.5	0.9583	1.0	1.0	3
cross	not_used	0.0	0.0	1.0	3
dense	not_used	0.75	0.75	1.0	3
e5	intfloat/e5-base-v2	0.9306	0.9583	1.0	3
lexical	not_used	0.75	0.75	1.0	3
neural	sentence-transformers/all-MiniLM-L6-v2	0.9722	0.9722	1.0	3

The answer-only experiment intentionally removes artifact identifiers and citations, so retrieval dependence is no longer credited unless it manifests in the text. This setting is weaker than gray-box RAG auditing, but it avoids assuming privileged access and gives practitioners a conservative deployment mode.

Formal guarantees. ADCU is not a replacement for certified removal, deletion-as-control, differential privacy, or pan-private continual release. Its role is complementary: it audits deployed pipeline behavior, derivative artifacts, caches, summaries, adapters, and retrieval-control layers that often sit outside the formal guarantee boundary. When a component has a formal deletion guarantee, ADCU should treat that guarantee as prior evidence and spend more probes on uncovered downstream artifacts.

9 ARTIFACT AND REPRODUCIBILITY

The artifact is organized as a reusable benchmark rather than a single script. The core package contains data classes for deletion targets, artifacts, probes, responses, evidence vectors, provenance graphs, risk scoring, audit algorithms, RAG harnesses, LoRA-SFT experiments, DenseRAG experiments, and reporting utilities. Scripts generate raw JSON/CSV rows, aggregate tables, SVG figures, and the manuscript PDF.

The main repeated commands are:

- `run_adcu_submission_experiments.py`: repeated RealRAG, NaturalFEVER, FineTuneSim, and HybridReal experiments.

- `run_adcu_advanced_experiments.py`: LoRA-SFT and DenseRAG experiments.
- `make_adcu_submission_tables.py` and `make_adcu_advanced_tables.py`: manuscript tables.
- `make_adcu_submission_figures.py`: benchmark figures.

The benchmark card specifies tracks, metrics, baselines, and release assumptions. Every exported result row includes the track, deletion method, audit method, decision, failure label, detection label, risk score, leakage channels, retain utility, audit calls, seed, target size, and budget where applicable. The advanced rows additionally include pretrained-LoRA forget margins, retain perplexity, retain completion accuracy, neural embedding model names, and top-*k* deleted-derivative hit rates. A requirements file, one-command reproduction script, and model-cache manifest are included with expected runtime and cache assumptions.

Code availability. The artifact code, benchmark harness, experiment scripts, generated tables, and manuscript sources are available at <https://github.com/LeoTTTPhat/ADCU>. The repository includes the one-command reproduction script, environment requirements, model-cache manifest, benchmark card, and redacted human-adjudication logs used to regenerate the reported results.

The current artifact uses synthetic private data and public FEVER evidence passages. This design avoids releasing sensitive records while still testing realistic deletion failure modes: index-only deletion, shadow-copy retention, cache

TABLE 15
Probe-family ablations with bootstrap confidence intervals.

Track	Probe Family	Failure Detection Rate	Bootstrap Ci95 Low	Bootstrap Ci95 High	N
DenseRAG	direct_only	0.3333	0.1111	0.5556	18
DenseRAG	paraphrase_only	0.6667	0.4444	0.8889	18
DenseRAG	retrieval_only	1.0	1.0	1.0	18
DenseRAG	extraction_only	0.0	0.0	0.0	18
DenseRAG	full_adcu	1.0	1.0	1.0	18
LoRA-SFT	direct_only	1.0	1.0	1.0	6
LoRA-SFT	paraphrase_only	1.0	1.0	1.0	6
LoRA-SFT	retrieval_only	0.0	0.0	0.0	6
LoRA-SFT	extraction_only	0.0	0.0	0.0	6
LoRA-SFT	full_adcu	1.0	1.0	1.0	6
PEFT-LoRA	direct_only	1.0	1.0	1.0	1
PEFT-LoRA	paraphrase_only	1.0	1.0	1.0	1
PEFT-LoRA	retrieval_only	0.0	0.0	0.0	1
PEFT-LoRA	extraction_only	0.0	0.0	0.0	1
PEFT-LoRA	full_adcu	1.0	1.0	1.0	1
Pretrained-LoRA	direct_only	0.0556	0.0	0.1667	18
Pretrained-LoRA	paraphrase_only	0.0556	0.0	0.1667	18
Pretrained-LoRA	retrieval_only	0.0	0.0	0.0	18
Pretrained-LoRA	extraction_only	0.0	0.0	0.0	18
Pretrained-LoRA	full_adcu	1.0	1.0	1.0	18
Qwen-TOFU-LoRA	direct_only	0.0	0.0	0.0	2
Qwen-TOFU-LoRA	paraphrase_only	1.0	1.0	1.0	2
Qwen-TOFU-LoRA	retrieval_only	0.0	0.0	0.0	2
Qwen-TOFU-LoRA	extraction_only	0.0	0.0	0.0	2
Qwen-TOFU-LoRA	full_adcu	1.0	1.0	1.0	2

TABLE 16
Redacted case studies from advanced experiments.

Case Id	Risk Ucb	Direct Leakage	Paraphrase Leakage	Retrieval Dependence	Interpretation
PEFT-LoRA::tiny HF PEFT LoRA validation::ADCU	0.7661	0.5437	0.6396	0.0	Residual dependence remains after the nominal deletion/unlearning operation.
Pretrained-LoRA::pretrained LoRA NPO unlearning::ADCU	0.4161	0.1548	0.1181	0.0	Residual dependence remains after the nominal deletion/unlearning operation.
LoRA-SFT::full adapter no unlearning::ADCU	0.9262	0.6012	0.6737	0.0	Residual dependence remains after the nominal deletion/unlearning operation.
DenseRAG::dense retrieval with retained derivatives::RetrieverHit	0.7858	0.2396	0.2726	0.375	Residual dependence remains after the nominal deletion/unlearning operation.
DenseRAG::e5 retrieval with retained derivatives::ADCU	0.7858	0.381	0.3796	0.4583	Residual dependence remains after the nominal deletion/unlearning operation.
DenseRAG::lexical retrieval with retained derivatives::ADCU	0.7858	0.2976	0.316	0.375	Residual dependence remains after the nominal deletion/unlearning operation.

retention, adapter memory, negative fine-tuning, gradient-ascent unlearning, synthetic derivative retention, and over-unlearning. The artifact should be interpreted as a technical audit benchmark, not as a legal compliance certification tool.

10 CONCLUSION

This paper frames foundation-model unlearning as an auditable data engineering problem. ADCU combines provenance graphs, valuation-guided probe allocation, black-box behavioral tests, and confidence-bounded deletion-risk reporting. The repeated RealRAG, FineTuneSim, HybridReal, LoRA-SFT, DenseRAG, and attack-suite results show that narrow exact-match, canary-only, retriever-only, membership-style, and extraction-only audits miss important residual-dependence channels. The resulting framework provides a concrete path toward reproducible deletion verification for RAG, fine-tuned, and hybrid foundation-model systems.

APPENDIX A BENCHMARK PROTOCOL DETAILS

This appendix provides the operational details needed to reproduce and extend ADCU-Bench. It is included in the draft because deletion-audit papers are easy to underspecify: small differences in deletion target construction, probe generation, cache handling, or retain-utility reporting can change the interpretation of results.

A.1 Deletion Target Construction

Each synthetic deletion target contains five elements. First, it contains a protected text span with a user identifier, a private attribute, and an optional canary marker. Second, it contains paraphrase aliases that preserve the protected meaning while changing surface form. Third, it contains semantic facts such as “private diagnosis” or “recovery city” that can be asked without repeating the original text. Fourth, it contains provenance links to raw chunks, summaries, synthetic instruction records, and caches. Fifth, it contains a pre-deletion influence score used for valuation-guided audit allocation.

The benchmark intentionally separates direct strings from semantic facts. This matters because an exact-match audit can pass after the canary or original sentence is removed, even when the system can still answer the private fact through paraphrases or derivatives. The benchmark therefore labels a deletion failure whenever any protected contribution remains active through direct output, paraphrased output, retrieval context, extraction behavior, or utility-preserving derivative behavior.

A.2 Probe Families

ADCU-Bench uses six probe families. Direct probes ask for the protected fact in a straightforward way. Paraphrase probes use aliases or alternate wordings. Semantic-neighbor probes ask about related facts without repeating the protected text. Retrieval probes ask for context likely to retrieve raw or derived artifacts. Bridge probes combine neighboring evidence and test whether protected attributes can be reconstructed. Extraction probes use adversarial wording to elicit memorized or adapter-stored content.

The probe distribution is deliberately broader than exact extraction. Deployed systems often leak by answering ordinary user questions rather than by dumping training records. A robust deletion audit should therefore include ordinary question answering, retrieval inspection, and adversarial extraction.

A.3 Deletion Methods

The RealRAG track includes provenance-guided deletion, index-only deletion, shadow-copy deletion, and cache-not-purged deletion. Provenance-guided deletion removes all artifacts linked to the target. Index-only deletion leaves summaries and caches active. Shadow-copy deletion leaves near-duplicates under different artifact identifiers. Cache-not-purged deletion leaves previous generated responses or retrieved contexts available.

The FineTuneSim and LoRA-SFT tracks include filtered retraining, gradient-ascent unlearning, negative SFT, adapter replacement failure, and no-unlearning baselines. Filtered retraining removes forget examples before adapter training. Gradient-ascent unlearning maximizes loss on forget examples while preserving retain loss. Negative SFT maps forget prompts to a no-information class. Adapter replacement failure models hidden or merged low-rank residues.

The HybridReal track combines both failure surfaces. A source document can be removed from RAG while synthetic examples or adapter behavior remain active; conversely, adapter unlearning can succeed while retrieved derivatives still reveal deleted information.

A.4 Audit Baselines

ExactMatch scores overlap with protected strings. Canary-Only checks whether explicit canary tokens remain. RetrieverHit checks whether deleted or derived artifact identifiers appear in retrieved contexts. MembershipInference scores direct and paraphrased behavioral dependence. ExtractionAudit emphasizes extraction probes. InfluenceProxy aggregates direct, paraphrase, retrieval, and counterfactual

components as a low-cost influence-style approximation. UniformProbes uses the same evidence channels as ADCU but allocates budget evenly. ADCU-NoValuation removes valuation-guided priority while keeping the rest of the audit.

These baselines are not straw men. They correspond to practical audit shortcuts: checking that vector IDs are gone, checking that canaries do not appear, checking exact text reproduction, or probing a fixed set of known prompts. The attack suite demonstrates why each shortcut can be insufficient.

A.5 Confidence Intervals

For row-level risk, ADCU uses a Hoeffding upper confidence bound because evidence scores are bounded in $[0, 1]$. For repeated detection rates, the artifact reports bootstrap 95% confidence intervals over binary failure-detection outcomes. It also exports per-seed variance tables so reviewers can separate average performance from seed-level instability.

A.6 Proofs for Risk Bounds

This appendix gives the complete proof for the confidence claims used in Section 4. Fix a post-deletion system S_- , a deletion target z , a declared audit-probe distribution $A(z)$, and a nonnegative weight vector w normalized so that $\|w\|_1 \leq 1$. For probe $q_i \sim A(z)$, define the scalar audit evidence

$$X_i(z) = w^\top e(S_-, q_i, z).$$

Because each evidence channel is in $[0, 1]$ and w is nonnegative with $\|w\|_1 \leq 1$, $X_i(z) \in [0, 1]$. The true target risk is

$$\mu_z = R_{\text{del}}(z; S_-) = \mathbb{E}[X_i(z)],$$

and the empirical risk over n_z probes is

$$\hat{\mu}_z = \frac{1}{n_z} \sum_{i=1}^{n_z} X_i(z).$$

Target-level false-pass control. Assume $X_1(z), \dots, X_{n_z}(z)$ are independent draws from the declared probe distribution. Hoeffding's inequality for bounded variables gives, for any $\epsilon > 0$,

$$\Pr[\mu_z - \hat{\mu}_z \geq \epsilon] \leq \exp(-2n_z\epsilon^2).$$

Set

$$\epsilon_z(\delta) = \sqrt{\frac{\log(1/\delta)}{2n_z}}.$$

Then $\exp(-2n_z\epsilon_z(\delta)^2) = \delta$, so

$$\Pr[\mu_z > \hat{\mu}_z + \epsilon_z(\delta)] \leq \delta.$$

Define $\text{UCB}(z) = \hat{\mu}_z + \epsilon_z(\delta)$. If the audit passes target z only when $\text{UCB}(z) \leq \tau$, then the false-pass event

$$\{\text{pass}(z) \wedge \mu_z > \tau\}$$

implies $\mu_z > \text{UCB}(z)$, because pass gives $\text{UCB}(z) \leq \tau$. Therefore

$$\Pr[\text{pass}(z) \wedge \mu_z > \tau] \leq \Pr[\mu_z > \text{UCB}(z)] \leq \delta.$$

This proves the target-level false-pass statement.

Multiple targets. For a deletion request D_{del} with targets $z \in D_{\text{del}}$, choose per-target confidence levels δ_z such that $\sum_z \delta_z \leq \delta_{\text{req}}$. Applying the target-level bound to each z and using the union bound gives

$$\Pr[\exists z : \mu_z > \text{UCB}(z)] \leq \sum_z \delta_z \leq \delta_{\text{req}}.$$

Thus, with probability at least $1 - \delta_{\text{req}}$, every target risk is upper-bounded by its reported UCB simultaneously.

Aggregate request-level risk. Let $\pi(z) \geq 0$ and $\sum_z \pi(z) = 1$. The true aggregate risk is

$$R_{\text{ADCU}}(D_{\text{del}}; S_-) = \sum_z \pi(z) \mu_z.$$

On the simultaneous-good event above, $\mu_z \leq \text{UCB}(z)$ for all targets, so

$$R_{\text{ADCU}}(D_{\text{del}}; S_-) = \sum_z \pi(z) \mu_z \leq \sum_z \pi(z) \text{UCB}(z).$$

Therefore, if the request-level audit passes only when

$$\sum_z \pi(z) \text{UCB}(z) \leq \tau_{\text{req}},$$

then

$$\Pr[\text{pass}(D_{\text{del}}) \wedge R_{\text{ADCU}}(D_{\text{del}}; S_-) > \tau_{\text{req}}] \leq \delta_{\text{req}},$$

again under the declared probe distributions and independence assumptions.

Sample complexity for detection above tolerance. Suppose a target has true risk $\mu_z \geq \tau + \gamma$ for $\gamma > 0$. To make the confidence radius at most $\gamma/2$, it suffices that

$$n_z \geq \frac{2 \log(1/\delta)}{\gamma^2},$$

because then $\epsilon_z(\delta) \leq \gamma/2$. Hoeffding also gives

$$\Pr[\hat{\mu}_z \leq \mu_z - \gamma/2] \leq \exp(-n_z \gamma^2 / 2) \leq \delta.$$

On the complementary event, $\hat{\mu}_z \geq \mu_z - \gamma/2 \geq \tau + \gamma/2$, so the empirical risk itself exceeds τ . This shows that $O(\log(1/\delta)/\gamma^2)$ probes are sufficient to detect a target whose risk is separated from the tolerance by margin γ , up to constants depending on whether one requires the empirical mean or the UCB radius to cross the operational fail threshold. The shorter bound in Section 4 corresponds to the condition that the UCB radius is at most γ ; the stronger display here gives a direct high-probability empirical separation statement.

Role of retain utility. The proof above controls deletion-risk false passes only. The actual audit also requires retain utility to exceed a threshold. This additional condition can only make passing harder. It therefore cannot increase the probability of passing while deletion risk exceeds tolerance, although it can convert otherwise passing runs into *escalate* decisions when unlearning damages retained behavior.

Assumptions and limits. The guarantee is conditional on the declared probe distribution, bounded evidence scores, stable post-deletion system behavior during the audit, and independence or martingale-style concentration for the sampled probes. It is not a universal proof that no prompt can ever elicit deleted information. If probes

are adaptively selected, the same proof can be recovered by replacing the i.i.d. Hoeffding step with an appropriate bounded martingale concentration bound, provided each next probe is chosen using only previous audit observations and the scoring variables remain bounded.

A.7 DenseRAG Configuration

DenseRAG uses TF-IDF features followed by truncated SVD as a local dense retriever. It also includes cache-gated neural retrieval with MiniLM sentence-transformer embeddings, E5-base-v2 retrieval, and BGE-small retrieval over all advanced seeds. E5 queries are prefixed with `query:` and documents with `passage:`; BGE queries use the standard retrieval instruction prefix. The cross-encoder-style reranker trains a logistic model over sparse similarity, dense similarity, and token-overlap features. Neural retrieval is limited to a small retain set in the artifact so reviewers can reproduce the run on CPU.

A.8 LoRA-SFT Configuration

The LoRA-SFT experiment uses a frozen base linear map and trainable low-rank matrices. Inputs are hashed token and character-ngram features. The model is trained with cross-entropy on retain examples and target-specific private examples. The Pretrained-LoRA experiment additionally loads a cached SmolLM2-135M safetensors checkpoint, wraps it with PEFT LoRA modules on attention projections, fine-tunes private completions, and compares filtered retraining, gradient ascent, negative SFT, NPO with a frozen reference adapter, and SimNPO without the reference correction. The Qwen-TOFU-LoRA experiment loads a cached Qwen2.5-0.5B checkpoint, uses TOFU `forget01` examples as public forget targets and TOFU `retain99` examples as retain examples, and compares full SFT with NPO on a compact CPU-reproducible slice. The full TOFU evaluator records all official TOFU split sizes, evaluates the full `forget01` split, and uses deterministic bounded subsets for the larger retain, holdout, real-author, world-fact, and perturbed splits in the default artifact run. Setting `ADCU_TOFU_EVAL_LIMIT=full` removes this cap and scores every example in every official split with the same Qwen likelihood metric. The audit converts protected-versus-refusal likelihood margins into an active-memory map before running the same black-box probe interface used by all other tracks. The artifact also reports retain perplexity and retain completion accuracy.

Table 17 reports the utility-risk view used in the reviewer-response revision. Retain utility is measured as FEVER evidence retrieval accuracy in RAG tracks, configured retain completion or accuracy in adapter tracks, or a deliberately low utility score for over-unlearning. Mean audit calls are reported as the black-box latency proxy; hardware-specific model latency is reported in the artifact manifest rather than folded into deletion risk.

A.9 Case-Study Redaction

Case studies are redacted before entering manuscript tables. The artifact stores risk components and interpretations but replaces synthetic user identifiers and secret values with

TABLE 17
Utility–risk summary for ADCU across repeated deletion cases.

Track	Deletion Method	Mean Risk Ucb	Mean Retain Utility	Mean Audit Calls	Failure Detection Rate
FineTuneSim	adapter replacement failure	0.829	0.965	25.33	1.0
FineTuneSim	filtered retraining	0.508	0.985	25.33	0.0
FineTuneSim	gradient-ascent unlearning	0.6716	0.965	25.33	1.0
FineTuneSim	negative fine-tuning	0.606	0.955	25.33	0.8889
FineTuneSim	over-unlearned adapter	0.508	0.42	25.33	1.0
HybridReal	RAG-only deletion	0.8045	0.98	25.33	1.0
HybridReal	adapter-only unlearning	0.8772	0.98	25.33	1.0
HybridReal	full provenance purge	0.508	0.98	25.33	0.0
HybridReal	graph-pivot expansion retained	0.877	0.98	25.33	1.0
HybridReal	synthetic derivative retained	0.9846	0.98	25.33	1.0
NaturalFEVER	answer-only paraphrase leak	0.8427	0.98	25.78	1.0
NaturalFEVER	citation cache retained	0.6985	0.98	25.78	1.0
NaturalFEVER	full provenance purge	0.5078	0.98	25.78	0.0
NaturalFEVER	natural summary retained	0.6632	0.98	25.78	1.0
RealRAG	backdoor-triggered RAG leakage	0.6716	0.98	25.33	1.0
RealRAG	cache-not-purged deletion	0.7155	0.98	25.33	1.0
RealRAG	index-only deletion	0.7155	0.98	25.33	1.0
RealRAG	provenance-guided deletion	0.508	0.98	25.33	0.0
RealRAG	shadow-copy deletion	0.6696	0.98	25.33	1.0

placeholders. This mirrors how a real deletion-audit report should be prepared: auditors need enough evidence to diagnose residual dependence, but papers and public artifacts should not normalize printing sensitive strings.

A.10 Recommended Extensions

The next version of ADCU-Bench should run unbounded Qwen likelihood evaluation on every retain and perturbed TOFU row, and add additional larger open checkpoints. It should also include separate gray-box and black-box settings: gray-box audits may inspect retrieved artifact identifiers, while black-box audits only observe the final answer. Reporting both settings will make the benchmark more useful to application teams with different levels of system access.

APPENDIX B ARTIFACT CHECKLIST

Environment. The artifact is tested with Python 3.11, PyTorch, scikit-learn, pandas, pyarrow, transformers, PEFT, and sentence-transformers. PEFT and sentence-transformers are optional at runtime. If a HuggingFace model or sentence-transformer checkpoint is not locally cached, the artifact records a fallback row rather than silently downloading a model. This behavior is deliberate: repeatable audit artifacts should not depend on unlogged network state.

One-command reproduction. A complete reproduction run executes the submission experiments, attack suite, advanced experiments, table generation, figure generation, tests, and manuscript build. The expected outputs are JSON/CSV result files, Markdown/LaTeX tables, PDF figures, and the final IEEE-style PDF. The result files are row-level rather than only aggregate-level so that reviewers can recompute summaries.

Data release. The public retain data comes from local FEVER evidence files. The private deletion targets are generated synthetically from seeds. The benchmark should

release the generation code and seeds rather than a fixed private-data dump. This makes it possible to regenerate targets, vary target counts, and avoid accidental normalization of sensitive-looking strings in public artifacts.

Audit reports. Each audit report includes the decision, aggregate risk UCB, per-target UCBs, empirical risk, priorities, number of probes, violating probe identifiers, retain utility, and notes. For paper tables, protected content is summarized through channel scores and redacted case-study rows.

Expected reviewer checks. Reviewers should be able to verify that exact-match, canary-only, retriever-hit, membership-inference, extraction, influence-proxy, uniform-probe, no-valuation, and full ADCU methods are all run through the same audit interface. They should also be able to verify that false alarms are computed on clean provenance-guided deletion cases and that retain utility is reported beside deletion risk.

APPENDIX C LIMITATIONS AND FAILURE CASES

Probe incompleteness. No finite probe suite can cover all possible prompts. ADCU reduces this risk by using multiple probe families, but its certificate is conditional on the declared probe distribution. If a future adversary uses an unseen language, modality, prompt style, or multi-turn strategy, additional probes may be needed.

Semantic scoring limits. The current evidence scorers combine lexical, semantic-token, retrieval, extraction, and watermark signals. This is intentionally lightweight and reproducible, but it can miss subtle paraphrases. A stronger submission version should add NLI or embedding-based semantic equivalence scoring with frozen, versioned judge models.

RAG metadata access. Some deployments expose retrieved context identifiers and citations; others expose only final answers. ADCU supports both settings, but retrieval-dependence evidence is stronger in gray-box RAG settings.

A purely black-box deployment should report wider uncertainty or rely more heavily on semantic and extraction probes.

Adapter opacity. Merged or quantized adapters can obscure provenance. The LoRA-SFT and PEFT paths test low-rank adaptation behavior, but they do not solve the general problem of auditing hidden downstream model merges. In such settings, the audit should treat the model as black-box and increase extraction, paraphrase, and counterfactual probe budgets.

Legal interpretation. ADCU is a technical audit framework. It can support deletion governance by producing repeatable evidence, but it does not determine whether a system satisfies GDPR, CCPA, contractual deletion requirements, or sector-specific privacy law. Legal compliance requires organizational context, policy interpretation, and human review.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Empirical Methods in Natural Language Processing*, 2020.
- [3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *European Chapter of the Association for Computational Linguistics*, 2021.
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [5] P. Voigt and A. von dem Bussche, *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer, 2017.
- [6] A. Cohen, A. Smith, M. Swanberg, and P. N. Vasudevan, "Control, confidentiality, and the right to be forgotten," *arXiv preprint arXiv:2210.07876*, 2022.
- [7] T. Fan, H. Gu, X. Cao, C. S. Chan, Q. Chen, Y. Chen, Y. Feng, Y. Gu, J. Geng, B. Luo, S. Liu, W. K. Ong, C. Ren, J. Shao, C. Sun, X. Tang, H.-X. Tae, Y. Tong, S. Wei, F. Wu, W. Xi, M. Xu, H. Yang, X. Yang, J. Yan, H. Yu, T. Zhang, Y. Zhang, X. Zhang, Z. Zheng, L. Fan, and Q. Yang, "Ten challenging problems in federated foundation models," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 7, pp. 4314–4337, 2025.
- [8] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," *IEEE Symposium on Security and Privacy*, 2015.
- [9] A. Ginart, M. Guan, G. Valiant, and J. Y. Zou, "Making ai forget you: Data deletion in machine learning," in *Advances in Neural Information Processing Systems*, 2019.
- [10] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *IEEE Symposium on Security and Privacy*, 2021.
- [11] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in *International Conference on Machine Learning*, 2020.
- [12] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, "Tofu: A task of fictitious unlearning for llms," in *arXiv preprint arXiv:2401.06121*, 2024.
- [13] Y. Yao, X. Xu, and Y. Liu, "Large language model unlearning," in *Advances in Neural Information Processing Systems*, 2024.
- [14] Z. Liu *et al.*, "Rethinking machine unlearning for large language models," *arXiv preprint arXiv:2402.08787*, 2024.
- [15] L. Wang *et al.*, "Machine unlearning in large language models," *arXiv preprint arXiv:2404.16841*, 2024.
- [16] Z. Jin *et al.*, "Digital forgetting in large language models: A survey of unlearning methods," *Artificial Intelligence Review*, 2025.
- [17] N. Carlini, C. Liu, U. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *USENIX Security Symposium*, 2019.
- [18] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *USENIX Security Symposium*, 2021.
- [19] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramer, and C. Zhang, "Quantifying memorization across neural language models," in *International Conference on Learning Representations*, 2023.
- [20] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE Symposium on Security and Privacy*, 2017.
- [21] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *IEEE Computer Security Foundations Symposium*, 2018.
- [22] J. Mattern, F. Miresghallah, Z. Jin, H. Schutze, M. Sachan, and T. Berg-Kirkpatrick, "Membership inference attacks against language models via neighbourhood comparison," *Findings of the Association for Computational Linguistics*, 2023.
- [23] Y. Gao *et al.*, "Retrieval-augmented generation for ai-generated content: A survey," *arXiv preprint arXiv:2402.19473*, 2024.
- [24] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," in *International Conference on Learning Representations*, 2024.
- [25] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, "Evaluation of retrieval-augmented generation: A survey," *arXiv preprint arXiv:2405.07437*, 2024.
- [26] R. Akkiraju, A. Xu, D. Bora, T. Yu, L. An *et al.*, "Facts about building retrieval augmented generation-based chatbots," *arXiv preprint arXiv:2407.07858*, 2024.
- [27] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference*, 2006.
- [28] Y. Wang, W. Qu, S. Zhai, Y. Jiang, Z. Liu, Y. Liu, Y. Dong, and J. Zhang, "Silent leaks: Implicit knowledge extraction attack on rag systems through benign queries," *arXiv preprint arXiv:2505.15420*, 2025.
- [29] Y. Xie, Y. Zhang, Y. Li, S. Song, X. Chen, Z. Liu, L. Su, and T. Liu, "Detecting rag extraction attack via dual-path runtime integrity game," *arXiv preprint arXiv:2604.10717*, 2026.
- [30] M. Song, J. Kim, M. Kang, H. Kim, S. Shin, and S. Son, "Subgraph reconstruction attacks on graph rag deployments with practical defenses," *arXiv preprint arXiv:2602.06495*, 2026.
- [31] S. Thornton, "Retrieval pivot attacks in hybrid rag: Measuring and mitigating amplified leakage from vector seeds to graph expansion," *arXiv preprint arXiv:2602.08668*, 2026.
- [32] A. Ghorbani and J. Zou, "Data shapley: Equitable valuation of data for machine learning," in *International Conference on Machine Learning*, 2019.
- [33] R. Jia, D. Dao, B. Wang, F. A. Hubis, N. Hynes, N. M. Gurel, B. Li, C. Zhang, D. Song, and C. J. Spanos, "Efficient task-specific data valuation for nearest neighbor algorithms," in *Proceedings of the VLDB Endowment*, 2019.
- [34] Y. Kwon and J. Zou, "Beta shapley: A unified and noise-reduced data valuation framework for machine learning," in *International Conference on Artificial Intelligence and Statistics*, 2022.
- [35] T. Wang and R. Jia, "Data banzhaf: A robust data valuation framework for machine learning," in *International Conference on Artificial Intelligence and Statistics*, 2023.
- [36] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *International Conference on Machine Learning*, 2017.
- [37] G. Pruthi, F. Liu, S. Kale, and M. Sundararajan, "Estimating training data influence by tracing gradient descent," in *Advances in Neural Information Processing Systems*, 2020.
- [38] V. Feldman and C. Zhang, "What neural networks memorize and why: Discovering the long tail via influence estimation," in *Advances in Neural Information Processing Systems*, 2020.
- [39] A. Ilyas, S. M. Park, L. Engstrom, G. Leclerc, and A. Madry, "Data-models: Predicting predictions from training data," in *International Conference on Machine Learning*, 2022.
- [40] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *International Conference on Machine Learning*, 2023.
- [41] X. Zhao *et al.*, "A survey of text watermarking in the era of large language models," *arXiv preprint arXiv:2312.07913*, 2023.

- [42] X. Wang, P. Sen, R. Li, and E. Yilmaz, "Adaptive retrieval-augmented generation for conversational systems," *arXiv preprint arXiv:2407.21712*, 2024.
- [43] H. Pham Van, N. M. Hieu, K. P. T. Tuan, N. L. Hai, L. N. Van, N. T. N. Diep, and T. Le, "Memorai: Memory organization and retrieval via adaptive graph intelligence for llm conversational agents," *arXiv preprint arXiv:2605.01386*, 2026.
- [44] Z. P. Lee, A. Lin, and C. Tan, "Finetune-rag: Fine-tuning language models to resist hallucination in retrieval-augmented generation," *arXiv preprint arXiv:2505.10792*, 2025.
- [45] R. Zhang, L. Lin, Y. Bai, and S. Mei, "Negative preference optimization: From catastrophic collapse to effective unlearning," in *Conference on Language Modeling*, 2024.
- [46] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.